

NAME: _____

BU ID: _____

Final Exam – Tuesday, December 17th, 2013 – 3pm – 7:30pm
EC327 Introduction to Software Engineering – Fall 2013

Total: 140 points

****MAKE SURE TO READ THE FOLLOWING BEFORE STARTING YOUR EXAM****

Exam Rules:

- **Save your work often during the exam.** You would not want to lose any work.
- You **CAN** use the following during the exam:
 - o Linux programs (g++, vi, gedit, gdb, emacs, and so on) installed on PHO305/7 machines,
 - o The Liang course textbook
- You **CANNOT** use hardware calculators, cell phones, laptops, any other electronic devices, other books, additional photocopied materials, or other printouts (not covered previously).
- You have until **5:00pm (first section) or 7:30pm (second session)** to finish and submit your exam. Please follow the submission instructions below. Your submission will have a timestamp, so make sure to submit before the deadline. **Late submissions will be penalized.**
- Do not spend too much time on any one question, you can always return to it later. **Focus on getting code that compiles and is well documented in the event that you cannot get the correct functionality. YOUR CODE MUST COMPILE OR YOU WILL BE GIVEN ZERO CREDIT THE PROBLEM.**
- **Question 0** is optional and is the same as the midterm and sample exams.
- Make sure to put all the code and all the files you need to submit in your exam directory, e.g., in *jbond_0007_002_final*. (See *Question 0* for more details about submission.)
- You are not allowed to access any other directories on the engineering grid (such as /ad/eng folders) or ssh to any machines. All actions on the lab machines are being logged during the exam.

GOOD LUCK!

Q0. Submission Procedure and Finding Your Way Around Linux.

*****This Question is Optional ** - This is the same procedure as the midterm***

- a. Open a terminal window.
- b. Create a folder named `<yourBUusername>_<last4digitsofBUID>_<machineNumber>_final`. For example, student *James Bond* with BU username *jbond*, BU ID *U12340007*, and machine number *002* should type:
`mkdir jbond_0007_002_final`
*** Make sure you put in your own name and number here, and not those of James Bond's!!!*
- c. Change directory to your final directory:
`cd jbond_0007_002_final`
- d. Create `test.cpp` file in your directory, and write the following code:

```
#include <iostream>
using namespace std;
int main()
{
    cout<< "TEST EC327 FINAL" << endl;
    return 0;
}
```
- e. Zip your directory and submit via the following steps:
`cd ..`
`ls`
*** You should see your jbond_0007_002_final directory listed now.*
`zip -r jbond_0007_002_final.zip ./jbond_0007_002_final/*`
*** Make sure to replace James Bond's folder name with your folder name.*
- f. Check that the zip file exists and contains all of the files:
`ls`
*** You should see jbond_0007_002_final.zip*
`less jbond_0007_002_final.zip`
*** This will list all the files in the zip. Make sure it contains all your files. ('q' to quit "less" program)*
- g. **Submit your solution:**
Submit your solution by uploading it to the exam website. Simply click "Browse" to select your file and then click "Upload File" to submit it.

You are allowed to submit your solutions as many times as you want; however, only the last submission will be graded. For each subsequent submission, please include a revision number in the filename so that we will know which submission is the latest (for example, `jbond_0007_002_final_rev2.zip`, `jbond_0007_002_final_rev3.zip`, etc.).

The upload system will prevent you from submitting a file with the same name as a file that has been already submitted. (This prevents you from overwriting previous submissions.) Hence the need to include revision numbers as discussed above. **Make sure the file size as reported by the submission system matches your expectations!**

Q1. Encapsulation [20 points] (15 minutes)

You are given a file `Q1.cpp` that compiles. However, many items are commented out and some functionality is missing. Your job is to add this functionality and uncomment the code.

Before beginning, make sure the `Q1.cpp` file compiles! (It should)

In class *Student* you need to:

1. Add two accessor functions for the two private variables *studied* and *iq*.

You need to make a class *Professor*. The code illustrates where this should be located. You need to:

1. Create a *willPass* function. This should take a *Student* as an argument and return a bool. Documentation is in the code for how it should perform.
2. Create a *addClass* function. If the *Student* “will pass” then you should increment the number of classes they have taken.

Finally uncomment the code in the *main* function. **DO NOT CHANGE** the *main* function other than to uncomment material.

Hint: Make sure to look at the output of your code to make sure it makes sense. Just because it compiles does not mean that it works. Do the student print statements make sense based on your code?

You should submit the following files for this problem:

`Q1.cpp`

Hints:

- You can compile this program using `g++ Q1.cpp`
- If you forget how compile the program under Linux, you can ask the staff for help so that you can continue with the exam, but it will cost you 5 points.

Q2. Inheritance Basics [15 points] (10 minutes)

Again, you are given a *Student* class (very similar to the last problem). This is in the file `Q2.cpp`. It should compile and run. **Make sure that it does compile BEFORE proceeding.**

You now need to add a *Slacker* class where indicated. This class should be derived from *Student*. It needs to have the following two components:

- A default constructor which sets *studied* to false, *iq* to half of a default *Student*, *classesTaken* to $\frac{1}{4}$ of a default *Student*, and *gpa* to $\frac{1}{5}$ of a default *Student*.
- Create a *setStudied* mutator function in *Slacker* which regardless of the input parameter always sets *studied* to false. Make sure this is called when generic programming is used. This may require a minor modification to *Student*.

When you are done, un-comment the lines in main and examine the output to see if it makes sense.

You should submit the following files for this problem:

`Q2.cpp`

Q3. Exceptions [20 points] (15 minutes)

Again this problem begins with a *Student* class. This is in the file `Q3.cpp`. It should compile and run. **Make sure that it does compile BEFORE proceeding.**

You need to create two classes:

1. *StudiesException* – this class must be derived from the C++ *exception* class. It needs to have a **public** member variable *sPtr* which is a pointer to a *Student*. In addition it needs a constructor which takes a pointer to a *Student* and assigns this to *sPtr*. In addition, this constructor should include the provided `cout` statement.

2. *Slacker* – this class must be derived from *Student*. It only needs to have a redefined *setStudied* function. This function should throw a *StudiesException* if called (Slackers don't study duh!).

In *main* you need to uncomment the lines provided. Here you need to add the required try and catch blocks as outlined in the code. **DO NOT add any other cout statements.**

You should submit the following files for this problem:
`Q3.cpp`

Hint: think if new header files need to be added. Work slowly and compile as you go to prevent having to compile the entire thing once all changes are made.

Q4. Operator Overloading and Dynamic Casting [25 points] (20 minutes)

In this problem you will overload the following operators: **+**, *****, **and** **/** for combining two *Students*. *Student* is an abstract base class and hence you have to use one of the two derived classes, *Slacker* and *MedStudent*.

This problem requires you to:

1. Create the following **non-member** overloaded operator functions:
 - a. `Student *operator+(Student &firstStudent, Student &secondStudent)`
 - i. This function should return a new *Slacker* **if either** argument is a *Slacker*. Otherwise return a *MedStudent*.
 - b. `Student *operator/(Student &firstStudent, Student &secondStudent)`
 - i. Should return a new Object of the type of the denominator.
 - c. `Student *operator*(Student &firstStudent, Student &secondStudent)`
 - i. This function should return a *Slacker* only **if both** arguments are *Slackers*. *MedStudent* otherwise.
2. Fix the 6 lines in the main function to perform the operations listed. **DO NOT CHANGE ANY OTHER LINES IN MAIN.**

IF YOU CHANGE MAIN TO HARDCODE THE SOLUTIONS YOU WILL LOSE ALL POINTS FOR THIS QUESTION.

This material is all in the file `Q4.cpp`. It should compile and run. **Make sure that it does compile BEFORE proceeding.**

You should submit the following files for this problem:

`Q4.cpp`

Q5. Templates and Arrays/Pointers [25 points] (20 minutes)

This problem requires you to make two functions with the following prototypes:

1. `T *destroy(T list[ ], int listSize, int &destroyedSize, T target)`
2. `T *replace(T list[ ], int listSize, T insert, T target)`

The destroy function needs to remove the target element(s) from the original list. The replace function needs to replace the target element(s) with the insert element.

More information is provided in the code. **If you have questions about the functionality please ask the staff before you begin.**

Both functions need be correct for full credit. The algorithmic complexity does not matter (i.e. just get it right). You will see if it is correct when you uncomment the main function code.

You should submit the following files for this problem:

`Q5.cpp`

Q6. STL Data Structures [25 points] (20 minutes)

In this question you will use the STL *stack* data structure to make a very simple mock up of the call stack.

The function prototypes for the STL *stack* class are in the provided code. You only potentially need to use these functions. **You do not need to use other functions or modify these.**

You are provided with a very simple class that acts as a *Function* object. You are also given a *stack* data structure template set to act as the stack for *Function* objects.

You need to add the following functions:

1. void call(string func_name, int val)
2. int ret()
3. void clearStack()

The *call* function should put a function on the stack creating a new *Function* object with the provided parameters. The *ret* function should return the value of the *Function* on top of the stack (and then remove the element). The *clearStack* function should print each of the *Function* names on the stack as it removes them (leaves the stack empty).

Addition comments/clarifications are in the code.

DO NOT MODIFY THE MAIN FUNCTION.

You should submit the following files for this problem:

Q6.cpp

Q7. Multiple Choice [10 points] (15 minutes)

Open Q7.txt. It currently lists 10 responses (all "c"). Modify this file **ONLY** by changing the response to the correct letter. **DO NOT** change the spacing or case of the letters.

1. Which is used to keep the call by reference value as intact?

- a) static
- b) const
- c) absolute
- d) none of the mentioned

2. What will happen in this code?

```
int a = 100, b = 200;  
int *p = &a, *q = &b;  
p = q;
```

- a) b is assigned to a
- b) p now points to b
- c) a is assigned to b
- d) q now points to a

3. In a class, member variables are often called its _____, and its member functions are sometimes referred to as _____.

- a) behavior, state
- b) none of these
- c) state, behavior
- d) data, activities
- e) values, methods

4. Irrespective of type of derivation _____ members of a base class are never accessible in derived class.

- a) (b), (d) and (e)
- b) private
- c) none of these
- d) protected
- e) public

5. When two or more classes serve as base class for a derived class, the situation is known as _____.

- a) multiple inheritance
- b) polymorphism
- c) encapsulation
- d) hierarchical inheritance
- e) none of these

6. When an object of a derived class is created

- a) derived class constructor is called followed by base class constructor
- b) base class constructor is called followed by derived class constructor
- c) base class constructor is not called at all
- d) none of the above

7. Evaluate $!(1 \ \&\& \ !(0 \ || \ 1))$

- a) True
- b) False
- c) Undecided
- d) 10

8. What does the class definitions in following code represent?

```
class Motorcycle
{
    Engine objEng;
};
class Engine
{
    float CC;
};
```

- a) kind of relationship
- b) has a relationship
- c) is a relationship
- d) Both A and B

9. What is the output of the following two options:

```
class A
{
    public:
        A(int n) : m_n(n)
        {
            if (0 == n) {
                throw std::logic_error("0");
            }
        }

        ~A() { std::cout << m_n; }

    public:
        const int m_n;
};

int main()
{
    A a(1); //OPTION #2
    try {
        A a(1); //OPTION #1
        A b(0);
        A c(2);
    }
    catch (const std::exception &) {
        std::cout << 3;
    }

    std::cout << std::endl;

    return 0;
}
```

- | | |
|--------------------|-----------------|
| a) Option 1 – “31” | Option 2 – “31” |
| b) Option 1 – “31” | Option 2 – “13” |
| c) Option 1 – “13” | Option 2 – “13” |
| d) Option 1 – “13” | Option 2 – “31” |

10. What is the output of the following code

```
class A
{
public:
    virtual ~A() {}
};

class B
    : public A
{
};

class C
    : public A
{
};

int main()
{
    try {
        B b;
        C c;
        A *pa1 = &b, *pa2 = &c;

        std::cout << ((dynamic_cast<B*>(pa1) != nullptr) ? 1 : 0);
        std::cout << ((dynamic_cast<B*>(pa2) != nullptr) ? 1 : 0);

        B &b1 = dynamic_cast<B&>(*pa2);
        pa1 = &b1;

        std::cout << ((static_cast<C*>(pa1) != nullptr) ? 1 : 0);
    }
    catch(const std::exception &) {
        std::cout << 2;
    }

    std::cout << std::endl;

    return 0;
}
```

- a) 001
- b) 110
- c) 102
- d) 112
- e) 012

End-of-Exam Checklist

- Your midterm folder should contain (case-sensitive):
 - o Q1.cpp
 - o Q2.cpp
 - o Q3.cpp
 - o Q4.cpp
 - o Q5.cpp
 - o Q6.cpp
 - o Q7.txt
- Zip your exam folder and submit it, following the procedure discussed in *Question 0*. (Remember that you'll need to use revision numbers on the filename if you've already submitted your exam.)
- **Before logging off, check with the staff to make sure that your exam has been received!**